

# ASP.NET Core Model Binding

*The Definitive Guide: What, Why, How, and The Unknowns*

## 1. What is Model Binding?

Model binding is the core ASP.NET Core subsystem responsible for acting as a bridge between the raw HTTP request and your C# application logic. It maps data from HTTP requests (which is inherently string-based or binary) to action method parameters (which are strongly-typed C# objects).

### The Core Philosophy

Instead of manually digging through HTTP contexts, parsing strings, and handling data conversion, Model Binding automatically hydrates your C# objects so you can focus strictly on business logic.

## 2. Why is it Needed? (Life Before Model Binding)

To understand its value, look at how you would handle an HTTP request *without* it:

```
// WITHOUT Model Binding (Manual Parsing)
public IActionResult CreateUser()
{
    var name = Request.Form["Name"].ToString();
    var ageString = Request.Query["Age"].ToString();

    if (!int.TryParse(ageString, out int age)) {
        return BadRequest("Invalid age");
    }

    var user = new User { Name = name, Age = age };
    // Process user...
}
```

With Model Binding, the framework handles the extraction, type conversion, and instantiation for you:

```
// WITH Model Binding
public IActionResult CreateUser(User user)
{
    // The framework already created and populated the 'user' object!
    // Process user...
}
```

## 3. How it Works: The Mechanics

---

When an endpoint is hit, the Model Binder goes through a specific lifecycle to find data.

### Value Providers

The framework searches for data in the HTTP request using **Value Providers**. By default, it looks in this exact order:

1. **Form fields** (Form data in the HTTP Body)
2. **Route data** (Variables in the URL path, e.g., `/api/users/{id}` )
3. **Query string parameters** (e.g., `?id=5&name=John` )

### Binding Sources (The `[From...]` Attributes)

You can override the default search order using attributes to explicitly tell the framework exactly where to look:

| Attribute      | Source in HTTP Request                              | Primary Use Case                                     |
|----------------|-----------------------------------------------------|------------------------------------------------------|
| [FromBody]     | Request Body (parsed by Input Formatters like JSON) | Receiving complex JSON objects in REST APIs.         |
| [FromQuery]    | Query String                                        | Filtering, sorting, and pagination parameters.       |
| [FromRoute]    | URL Route Parameters                                | Identifying specific resources (e.g., IDs).          |
| [FromForm]     | Form-urlencoded or multipart/form-data              | HTML form submissions and file uploads.              |
| [FromHeader]   | HTTP Headers                                        | Reading custom tokens, API keys, or correlation IDs. |
| [FromServices] | Dependency Injection Container                      | Injecting services directly into the action method.  |

## 4. The "Things You Didn't Mention" (Advanced Concepts)

Many developers use model binding daily without understanding these critical underlying concepts and edge cases.

### A. Model Binding vs. Input Formatters (The JSON Confusion)

#### Crucial Distinction

Model Binding (using Value Providers) and JSON Deserialization are **not the same mechanism**.

When you use [FromQuery] or [FromForm], the standard Model Binding system parses the data piece-by-piece. However, when you use [FromBody], the Model Binding system immediately delegates the entire process to an **Input Formatter** (usually `System.Text.Json`). This means behavior like date parsing or case-sensitivity depends entirely on your JSON settings, not your model binder settings.

## B. Overposting / Mass Assignment (Security Vulnerability)

If you bind directly to your database Entity models, users can send malicious data to overwrite fields you didn't intend to expose. For example, if your `User` class has an `IsAdmin` property, a user could send `?IsAdmin=true` in the query string, and the model binder will blindly set it to true.

**The Fix:** Always use **DTOs (Data Transfer Objects)** that only contain the exact properties you want the user to be able to submit.

## C. The `[BindRequired]` Attribute

By default, if a parameter is missing from the HTTP request, Model Binding silently assigns it a default value (e.g., `null` for strings, `0` for ints). If you want Model Binding to fail and add a Model Error when data is missing, use `[BindRequired]` (for non-body data).

## D. Complex Type Prefixing

If you are binding a complex object from a query string, the framework expects the query keys to match the property names. E.g. for a `SearchModel { int Page, string Term }`, the query is `?Page=1&Term=abc`. If your parameter is named, e.g., `Get(SearchModel search)`, you might sometimes need to prefix it like `?search.Page=1` depending on your configuration.

# 5. Custom Model Binders

---

Sometimes the built-in binders aren't enough. For example, you want to pass a comma-separated list in a query string (`?ids=1,2,3`) and have it automatically bind to an `int[]`. You can write a custom `IModelBinder`.

```

public class IntArrayModelBinder : IModelBinder
{
    public Task BindModelAsync(ModelBindingContext bindingContext)
    {
        // 1. Get the value from the value provider (e.g., the query string)
        var valueProviderResult = bindingContext.ValueProvider
            .GetValue(bindingContext.ModelName);

        if (valueProviderResult == ValueProviderResult.None)
            return Task.CompletedTask;

        var value = valueProviderResult.FirstValue;

        // 2. Parse the comma-separated string
        if (!string.IsNullOrEmpty(value))
        {
            var intArray = value.Split(',')
                .Select(int.Parse)
                .ToArray();

            // 3. Set the result!
            bindingContext.Result = ModelBindingResult.Success(intArray);
        }

        return Task.CompletedTask;
    }
}

```

You apply it to your model or parameter like this:

```

public IActionResult GetUsers([ModelBinder(typeof(IntArrayModelBinder))] int[] ids)

```

## 6. Best Practices & Common Pitfalls

- **Only one `[FromBody]` per action:** The HTTP request body is a forward-only stream. Once it is read by the Input Formatter, it cannot be read again. You cannot have two `[FromBody]` parameters in a single method.
- **Don't mix body and form:** You cannot use `[FromBody]` and `[FromForm]` on the same endpoint easily, because they dictate completely different Content-Types (application/json vs multipart/form-data).
- **Check `ModelState.IsValid` (Legacy MVC):** Model binding handles type conversion, but if it fails (e.g., someone typed "abc" into an integer field), it doesn't crash. It sets the property to 0

and adds an error to `ModelState`. If you aren't using `[ApiController]`, you *must* check `if (!ModelState.IsValid)` manually.